BookSpot — Roadmap v2 (Backend + Frontend, Vertical Slice)

Stack: React + TypeScript + Vite · Node.js + TypeScript + Express · PostgreSQL · Prisma · Redis · BullMQ
Status awal: Login, register, dan auth sudah selesai.

Cara pakai

- **Vertical slice:** selesaikan modul backend (+ test + OpenAPI) → langsung buat layar frontend modul itu → lanjut modul berikutnya. Frontend boleh tertinggal maksimal **satu sprint** dari backend.
- **ID card:** `Card n` = card dari roadmap lama · `N#` = card baru backend/infra · `F#` = card frontend.
- ➕ = item tambahan hasil review (celah logika / best practice).
- Satu sprint ≈ 1–2 minggu kerja paruh waktu.

Definition of Done (berlaku untuk SETIAP card)

- ☐ Validasi input (Zod) dan otorisasi sudah ada
- ☐ Error memakai format global yang konsisten
- ☐ Test (unit/integration) lulus dan CI hijau
- ☐ OpenAPI diperbarui
- ☐ Commit memakai konvensi (`feat:` , `fix:` , `test:` , `docs:` , `chore:`)

Tambahan untuk card F# yang mengambil data:

- ☐ Data server memakai `useQuery` dengan `queryKey` konsisten dan `staleTime` sesuai panduan di bawah
- ☐ Setelah mutasi, cache terkait di- `invalidate`
- ☐ Tidak ada data server yang disimpan di `useState` , `localStorage` , atau context

Panduan cache frontend (TanStack Query)

Data dari server di-cache oleh **TanStack Query**. `React.memo` , `useMemo` , dan `useCallback` bukan alat cache: pakai hanya untuk render ulang yang terbukti lambat (ukur dulu dengan React Profiler).

Data	staleTime	Catatan
Daftar dan detail venue	5 menit	Berubah hanya saat Owner mengedit
Resource dan jam operasional	5 menit	Invalidate setelah Owner mengubah
Availability	10 detik	Plus `refetchOnWindowFocus`
Booking saya, status pembayaran	0	Pakai refetch/polling
Dashboard Owner	1 menit	Agregasi berat

Default QueryClient (`lib/query-client.ts`):

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 60_000, retry: 1, refetchOnWindowFocus: false },
  },
});
```

Progress

Fase	Sprint	Isi	Status
A — MVP	0	Fondasi	[ ]
A — MVP	1	Venue	[ ]
A — MVP	2	Resource & Availability	[ ]
A — MVP	3	Booking inti	[ ]
B — Pembeda	4	Concurrency	[ ]
B — Pembeda	5	Payment	[ ]
B — Pembeda	6	Jobs & Notifikasi	[ ]
B — Pembeda	7	Dashboard & Review (opsional)	[ ]
C — Delivery	8	Hardening	[ ]
C — Delivery	9	Kualitas & Load Test	[ ]
C — Delivery	10	Deployment & Dokumentasi	[ ]

FASE A — MVP

Sprint 0 — Fondasi

N1 — TEST — Infrastruktur Test

- [ ] Install Vitest dan Supertest
- [ ] Konfigurasi `.env.test`
- [ ] Database test terisolasi (service/port terpisah)
- [ ] Script reset database antar test
- [ ] Factory data (user, venue, resource, booking)
- [ ] Helper token login per role (CUSTOMER, OWNER, ADMIN)
- [ ] Script `test`, `test:watch`, `test:int`

N2 — CORE — Logging & Error Handling

- [ ] pino + pino-http dengan request ID (`X-Request-Id`)
- [ ] Redact data sensitif (password, token, header authorization)
- [ ] Hierarki `AppError` (400, 401, 403, 404, 409, 422, 429)
- [ ] Global error handler: `{ error: { code, message, details, requestId } }`
- [ ] Map error Zod ke 422 dengan detail per field
- [ ] Map error Prisma (P2002 → 409, P2025 → 404)
- [ ] Test format error

N3 — CI — GitHub Actions

- [ ] Workflow: install, lint, typecheck, test
- [ ] Service Postgres dan Redis di CI
- [ ] `prisma migrate deploy` di CI
- [ ] Cache npm
- [ ] Badge status di README

N4 — DOCS — OpenAPI Foundation

- [ ] Pilih pendekatan (zod-to-openapi atau swagger-jsdoc)
- [ ] Sajikan `/api/docs`
- [ ] Dokumentasikan endpoint auth
- [ ] Export `openapi.json`
- [ ] Script generate tipe frontend (`openapi-typescript`)

N5 — DATA — Seed Dasar

- [ ] Konfigurasi `prisma db seed`
- [ ] Akun demo: admin, owner, customer
- [ ] Password dari env
- [ ] Seed bersifat idempotent (upsert)

N6 — AUTH — Audit Auth yang Sudah Jadi

- [ ] ➕ `role` tidak bisa dikirim client saat register
- [ ] ➕ Refresh token rotation dan revoke berfungsi
- [ ] ➕ Parameter hash password sesuai rekomendasi
- [ ] ➕ Test `requireAuth` dan `requireRole`
- [ ] ➕ Catat kebutuhan rate limit login (dikerjakan di Sprint 8)

F1 — FE — App Shell

- [ ] React Router dengan layout publik dan dashboard
- [ ] Setup Tailwind (shadcn/ui opsional)
- [ ] Komponen dasar: Button, Input, Select, Modal, Toast, Skeleton, EmptyState, Pagination
- [ ] Pola loading dan error global
- [ ] Konfigurasi proxy/env API

F2 — FE — API Client & Session

- [ ] API client bertipe dari OpenAPI
- [ ] Putuskan strategi token (rekomendasi: access token di memori, refresh token httpOnly cookie)
- [ ] Interceptor auto-refresh saat 401 (single-flight)
- [ ] `useAuth` / AuthContext
- [ ] `ProtectedRoute` dan `RoleGuard`
- [ ] Logout membersihkan cache query
- [ ] ➕ Cache: pastikan membungkus aplikasi di main.tsx
- [ ] ➕ Cache: atur default QueryClient di lib/query-client.ts: staleTime 60 detik, retry 1, refetchOnWindowFocus false
- [ ] ➕ Cache: buat query key factory per domain (venueKeys, resourceKeys, availabilityKeys, bookingKeys) di satu tempat
- [ ] ➕ Cache: data dari server tidak disimpan di useState, localStorage, atau context; selalu lewat useQuery

F3 — FE — Halaman Auth

- [ ] Register dan login (react-hook-form + Zod)
- [ ] Tampilkan error server per field

- [ ] Redirect setelah login sesuai role
- [ ] Halaman profil minimal

Selesai jika: login lewat UI berfungsi, route Owner/Admin terproteksi, CI hijau.

Sprint 1 — Venue

Card 1 — VENUE — Schema & Database

- [ ] Review model Venue
- [ ] Field Venue
- [ ] Enum status: `DRAFT`, `PENDING_APPROVAL`, `ACTIVE`, `REJECTED`, `INACTIVE`
- [ ] Type/category venue
- [ ] Relasi Owner
- [ ] Field alamat/lokasi (lat/lng opsional)
- [ ] Timestamps
- [ ] ➕ `deletedAt` (soft delete)
- [ ] ➕ `rejectionReason` (nullable)
- [ ] ➕ `timezone` (default `Asia/Jakarta`)
- [ ] ➕ Index `(city, status, type)`
- [ ] Migration
- [ ] Verifikasi schema

Card 2 — VENUE — CRUD API

- [ ] Zod schema, service, controller, routes
- [ ] `POST /api/venues`
- [ ] `GET /api/venues/:id`
- [ ] `PATCH /api/venues/:id`
- [ ] `DELETE /api/venues/:id`
- [ ] Otorisasi Owner dan Admin
- [ ] Owner tidak bisa mengubah venue Owner lain
- [ ] Handle not found dan input invalid
- [ ] ➕ Publik hanya bisa melihat venue `ACTIVE`; Owner/Admin melihat miliknya
- [ ] ➕ DELETE = soft delete; tolak jika ada booking mendatang (aktifkan di Sprint 3)
- [ ] Test CRUD

Card 3 — VENUE — Search & Filtering

- [ ] `GET /api/venues`
- [ ] Pagination (➕ batas `pageSize` maksimum)

- [ ] Search nama
- [ ] Filter type, city, status
- [ ] Sort
- [ ] Validasi query parameter
- [ ] ➕ Filter status dibatasi per role
- [ ] ➕ Cek query dengan `EXPLAIN ANALYZE`
- [ ] Test pagination, filter, dan kombinasi filter
- [ ] Geolocation (lat, lng): **opsional, jangan dipaksa**

Card 4 — VENUE — Approval & Status

- [ ] Definisikan transisi status
- [ ] `DRAFT → PENDING_APPROVAL`
- [ ] `PENDING_APPROVAL → ACTIVE`
- [ ] `PENDING_APPROVAL → REJECTED`
- [ ] `ACTIVE → INACTIVE`
- [ ] ➕ `REJECTED → DRAFT` (ajukan ulang)
- [ ] ➕ `INACTIVE → ACTIVE`
- [ ] Owner submit approval; Admin approve/reject (➕ dengan alasan)
- [ ] Cegah transisi tidak valid
- [ ] ➕ Putuskan: edit venue `ACTIVE` perlu re-approval atau tidak (catat sebagai ADR)
- [ ] ➕ Catat dampak `INACTIVE` ke booking yang ada (implementasi di Sprint 3)
- [ ] Test otorisasi dan transisi

F4 — FE — Daftar Venue Publik

- [ ] Halaman daftar dengan kartu venue
- [ ] Pencarian, filter, sort
- [ ] Pagination sinkron dengan URL query
- [ ] State loading, kosong, error
- [ ] ➕ Cache: useQuery dengan venueKeys.list(filter); filter dan page ikut menjadi bagian queryKey
- [ ] ➕ Cache: staleTime 5 menit dan placeholderData: keepPreviousData agar ganti halaman/filter tidak berkedip kosong

F5 — FE — Owner: Kelola Venue

- [ ] Daftar venue saya dengan badge status
- [ ] Form buat/ubah venue
- [ ] Tombol submit approval
- [ ] Tampilkan alasan penolakan

- [ ] ➕ Cache: daftar venue milik Owner memakai key terpisah (venueKeys.mine)
- [ ] ➕ Cache: invalidateQueries venueKeys setelah buat, ubah, hapus, dan submit approval

F6 — FE — Admin: Approval Venue

- [ ] Antrean venue `PENDING_APPROVAL`
- [ ] Approve dan reject (dengan alasan)
- [ ] ➕ Cache: query antrean approval dengan staleTime pendek (30 detik)
- [ ] ➕ Cache: invalidateQueries antrean dan venueKeys setelah approve/reject

Selesai jika: Owner membuat venue → Admin approve → venue tampil di daftar publik.

Sprint 2 — Resource & Availability

Card 5 — RESOURCE — Schema & Database

- [ ] Review model Resource
- [ ] Relasi Venue → Resource
- [ ] Resource type, capacity, status, timestamps
- [ ] Pricing
- [ ] ➕ Harga disimpan sebagai `Int` (rupiah), bukan float
- [ ] ➕ `deletedAt` (soft delete)
- [ ] Migration dan verifikasi relasi

Card 6 — RESOURCE — CRUD API

- [ ] `resource.schema.ts` , service, controller, routes
- [ ] `POST /api/venues/:venueId/resources`
- [ ] `GET /api/venues/:venueId/resources`
- [ ] `GET /api/resources/:id`
- [ ] `PATCH /api/resources/:id`
- [ ] `DELETE /api/resources/:id`
- [ ] Otorisasi Owner dan Admin
- [ ] Cek kepemilikan melalui Venue
- [ ] Validasi dan error handling
- [ ] Test CRUD

Card 7 — RESOURCE — Pricing & Metadata

- [ ] Pricing, capacity, status, description, metadata
- [ ] Validasi harga dan kapasitas

- [ ] ➕ Putuskan model harga: flat vs weekday/weekend (catat sebagai ADR)
- [ ] Test konfigurasi

Card 8 — AVAILABILITY — Operating Hours

- [ ] Model jam operasional
- [ ] Jam buka, jam tutup, hari aktif
- [ ] Granularitas slot (➕ 30 atau 60 menit, configurable)
- [ ] ➕ Jam operasional memakai timezone venue
- [ ] ➕ Jam melewati tengah malam: putuskan didukung atau ditolak validasi
- [ ] Konfigurasi oleh Owner dan Admin
- [ ] Validasi dan test

Card 9 — AVAILABILITY — Slot Generator

- [ ] Service slot generator
- [ ] Generate slot harian
- [ ] Patuhi jam buka, jam tutup, dan durasi slot
- [ ] Handle boundary time (➕ tentukan: jam tutup eksklusif)
- [ ] Handle jam operasional invalid
- [ ] ➕ Test konversi timezone (UTC ↔ WIB/WITA/WIT)
- [ ] Unit test slot generation

Card 10 — AVAILABILITY — Availability API

- [ ] `GET /api/resources/:id/availability?date=`
- [ ] Validasi parameter tanggal
- [ ] Ambil jadwal, booking existing, dan blocked slot
- [ ] Hitung slot tersedia dan tidak tersedia
- [ ] Response terstruktur
- [ ] Test slot available, booked, dan blocked

Card 11 — AVAILABILITY — Resource Blocking

- [ ] Blokir maintenance
- [ ] Hari libur/tutup
- [ ] Block dan unblock slot
- [ ] Otorisasi Owner dan Admin
- [ ] Cegah block invalid
- [ ] ➕ Cegah block yang menimpa booking aktif
- [ ] Test availability dengan blok

F7 — FE — Owner: Kelola Resource

- ☐ Daftar dan form resource per venue
- ☐ Atur harga dan status
- ☐ ➕ Cache: query resource per venue dengan staleTime 5 menit
- ☐ ➕ Cache: invalidateQueries resourceKeys dan detail venue setelah tambah/ubah/hapus resource

F8 — FE — Owner: Jam Operasional & Blokir

- ☐ Form jam operasional per hari
- ☐ Kalender blokir slot/tanggal
- ☐ ➕ Cache: query jam operasional dan blokir dengan staleTime 5 menit
- ☐ ➕ Cache: invalidateQueries jam operasional dan availabilityKeys setelah jam atau blokir diubah

F9 — FE — Detail Venue & Slot Picker

- ☐ Halaman detail venue + daftar resource
- ☐ Kalender tanggal
- ☐ Grid slot (tersedia / terisi / diblokir)
- ☐ Waktu ditampilkan sesuai timezone venue
- ☐ ➕ Cache: detail venue staleTime 5 menit; availability staleTime 10 detik dengan refetchOnWindowFocus
- ☐ ➕ Cache: availability memakai enabled: Boolean(resourceId && date) agar tidak request sebelum dipilih
- ☐ ➕ Cache: React.memo/useCallback pada SlotButton hanya jika React Profiler menunjukkan lag; ukur dulu

Selesai jika: pelanggan melihat slot kosong yang akurat; Owner bisa memblokir slot.

Sprint 3 — Booking Inti

Card 12 — BOOKING — State Machine & Schema

- ☐ Enum: `PENDING`, `CONFIRMED`, `CANCELLED`, `EXPIRED`, `COMPLETED`
- ☐ Definisikan transisi valid
- ☐ Model Booking, relasi User dan Resource
- ☐ `startTime` / `endTime`
- ☐ **Price snapshot** (harga saat booking dibuat)
- ☐ Timestamps

- ☐ ➕ `expiresAt`
- ☐ ➕ `cancelledAt`, `cancelledBy`, `cancelReason`
- ☐ ➕ Index `(resourceId, startTime, endTime)` dan `(userId, status)`
- ☐ Migration

Card 13 — BOOKING — Create & Retrieve

- ☐ Zod schema, service, controller, route
- ☐ `POST /api/bookings`
- ☐ `GET /api/bookings/:id`
- ☐ `GET /api/bookings/me`
- ☐ Verifikasi user, resource ada, dan resource aktif
- ☐ ➕ Venue harus `ACTIVE`
- ☐ ➕ Waktu tidak di masa lalu
- ☐ ➕ Berada dalam jam operasional dan sesuai granularitas slot
- ☐ ➕ Durasi minimum/maksimum (mis. maks 3 jam)
- ☐ ➕ Batas hari ke depan (mis. 30 hari)
- ☐ ➕ Cek overlap level aplikasi (lapisan pertama; constraint DB di Sprint 4 sebagai lapisan terakhir)
- ☐ ➕ Owner tidak bisa membooking resource miliknya lewat jalur Customer
- ☐ Hitung harga dan simpan snapshot
- ☐ Test sukses dan request invalid

Card 14 — BOOKING — Cancellation & Owner Management

- ☐ Cancel booking
- ☐ Aturan pembatalan Customer dan Owner
- ☐ Cegah pembatalan tidak sah dan pada state invalid
- ☐ List booking per venue
- ☐ Otorisasi Owner dan Admin
- ☐ ➕ Putuskan kebijakan pembatalan/refund: diimplementasikan atau *out of scope* (catat di ADR)
- ☐ ➕ Venue `INACTIVE`: tolak booking baru, tentukan nasib booking lama
- ☐ ➕ Aktifkan guard "tolak hapus venue/resource jika ada booking mendatang"
- ☐ Test pembatalan

F10 — FE — Alur Booking

- ☐ Pilih slot → ringkasan harga → konfirmasi
- ☐ Tampilan error validasi dari server

- ☐ ➕ Cache: useMutation buat booking; onSuccess invalidateQueries availabilityKeys resource terkait dan bookingKeys

F11 — FE — Booking Saya

- ☐ Daftar dan detail booking
- ☐ Badge status
- ☐ Tombol batalkan dengan konfirmasi
- ☐ ➕ Cache: query booking saya dengan staleTime 0
- ☐ ➕ Cache: invalidateQueries bookingKeys dan availabilityKeys setelah pembatalan

F12 — FE — Owner: Booking Masuk

- ☐ Daftar booking per venue (filter status/tanggal)
- ☐ Tampilan kalender
- ☐ Aksi pembatalan oleh Owner
- ☐ ➕ Cache: query booking per venue dengan staleTime 30 detik; filter status/tanggal masuk queryKey
- ☐ ➕ Cache: invalidateQueries bookingKeys dan availabilityKeys setelah Owner membatalkan

Selesai jika: alur Customer dari memilih slot hingga booking `PENDING` berjalan penuh di UI.

FASE B — PEMBEDA PORTOFOLIO

Sprint 4 — Concurrency

> Jangan dikerjakan sekaligus. Pahami *kenapa* tiap lapisan dibutuhkan sebelum menulis kodenya.

Card 15 — BOOKING — Database Transaction

- ☐ Identifikasi batas transaksi
- ☐ Prisma transaction untuk create booking secara atomik
- ☐ Handle kegagalan transaksi dan verifikasi rollback
- ☐ ➕ Di dalam transaksi: expire booking `PENDING` yang basi pada slot yang sama (lazy expire)
- ☐ Test kegagalan transaksi

Card 16 — BOOKING — Perlindungan Double Booking

- ☐ Definisikan time range booking
- ☐ Investigasi tipe range PostgreSQL (`tstzrange`)

- ☐ ➕ Aktifkan extension `btree_gist` lewat migration SQL manual
- ☐ Exclusion constraint pada `(resourceId, range)`
- ☐ ➕ Constraint **parsial**: hanya berlaku untuk `PENDING` dan `CONFIRMED`
- ☐ ➕ Pastikan `prisma migrate dev` tidak mendeteksi drift akibat constraint manual
- ☐ Tangkap error constraint (➕ SQLSTATE `23P01`) → HTTP 409 Conflict
- ☐ Test booking overlap dan non-overlap
- ☐ ➕ Test: booking `CANCELLED` / `EXPIRED` tidak memblokir slot

Card 17 — BOOKING — Idempotency

- ☐ Definisikan header `Idempotency-Key` dan validasinya
- ☐ Simpan request idempotency, kaitkan dengan user
- ☐ ➕ Unique constraint `(userId, key)`
- ☐ ➕ Simpan hash payload; key sama dengan payload berbeda → 422
- ☐ ➕ Tangani request yang masih diproses (in-flight)
- ☐ Request ulang mengembalikan hasil sebelumnya
- ☐ Cegah booking duplikat
- ☐ TTL / kedaluwarsa key
- ☐ Test request duplikat

Card 18 — BOOKING — Redis Distributed Lock (evaluasi)

- ☐ Evaluasi: apakah lock diperlukan di atas exclusion constraint?
- ☐ ➕ Tulis keputusan sebagai ADR (kebenaran data sudah dijamin constraint; lock = optimasi contention)
- ☐ Jika dipakai: lock key, TTL, acquire, release, handle gagal dan expire
- ☐ Test concurrent request dengan dan tanpa lock

Card 19 — BOOKING — Aturan Expiration

- ☐ Umur booking `PENDING` (mis. 15 menit)
- ☐ Kondisi expiration dan transisi yang diizinkan
- ☐ Perilaku setelah expired
- ☐ ➕ Availability memperlakukan `PENDING` lewat `expiresAt` sebagai slot kosong
- ☐ Test aturan expiration

Card 20 + 41 — BOOKING — Concurrency Test (digabung)

- ☐ Siapkan database test terisolasi, satu resource, satu slot
- ☐ Kirim 100 request paralel
- ☐ Tangkap request sukses dan ditolak
- ☐ Verifikasi: **1 sukses, 99 konflik**

- ☐ Verifikasi tidak ada booking duplikat dan database konsisten
- ☐ Test idempotency (dan Redis lock jika dipakai)
- ☐ Catat hasil untuk README

F13 — FE — UX Konflik & Idempotency

- ☐ Pesan "slot baru saja diambil" + refetch availability
- ☐ `Idempotency-Key` per percobaan submit
- ☐ Tombol submit dinonaktifkan saat proses
- ☐ Penanganan error 409 dan 422
- ☐ ➕ Cache: saat respons 409, invalidateQueries availabilityKeys resource terkait supaya slot yang sudah diambil hilang

Selesai jika: hasil concurrency test terdokumentasi dan UX konflik bekerja.

Sprint 5 — Payment

Card 21 — PAYMENT — Invoice & Status

- ☐ Model Payment, status pembayaran, relasi Booking
- ☐ Buat invoice dan payment reference
- ☐ Hitung amount (dari price snapshot)
- ☐ Lacak expiration
- ☐ Endpoint status pembayaran
- ☐ Test lifecycle pembayaran

Card 22 — PAYMENT — Mock Gateway

- ☐ ➕ Interface `PaymentProvider` agar bisa ditukar (mock ↔ Midtrans/Xendit sandbox)
- ☐ Mock provider: generate payment, return reference
- ☐ Simulasi sukses, gagal, dan expired
- ☐ Test tiap skenario

Card 23 — PAYMENT — Webhook

- ☐ `POST /api/payments/webhook`
- ☐ Validasi payload
- ☐ ➕ Verifikasi autentisitas: HMAC dengan shared secret + perbandingan timing-safe
- ☐ ➕ Tabel event webhook dengan `providerEventId` unik
- ☐ Proses event, update status payment dan booking
- ☐ Handle webhook duplikat (idempotent)

- ☐ ➕ Handle event tidak berurutan
- ☐ ➕ Update kondisional (`WHERE status = 'PENDING'`) di dalam transaksi
- ☐ ➕ Putuskan perilaku "bayar sukses setelah booking expired" (konfirmasi ulang jika slot kosong, atau tandai butuh refund)
- ☐ Test webhook sukses, gagal, dan duplikat

F14 — FE — Checkout

- ☐ Halaman checkout dengan ringkasan invoice
- ☐ Simulator gateway (tombol sukses / gagal / expired)
- ☐ ➕ Cache: query invoice dan status pembayaran dengan staleTime 0
- ☐ ➕ Cache: setelah simulasi pembayaran, invalidateQueries payment, bookingKeys, dan availabilityKeys

F15 — FE — Status Pembayaran

- ☐ Hitung mundur kedaluwarsa
- ☐ Status pembayaran dan polling/refetch
- ☐ Halaman invoice
- ☐ ➕ Cache: refetchInterval hanya saat status PENDING; berhenti otomatis saat status final (CONFIRMED/EXPIRED/CANCELLED)

Selesai jika: booking → bayar (mock) → webhook → `CONFIRMED` terlihat di UI.

Sprint 6 — Jobs & Notifikasi

> Urutan penting: email service (Card 28–29) dikerjakan **sebelum** job notifikasi (Card 26).

Card 24 — JOBS — Infrastruktur BullMQ

- ☐ Install BullMQ, koneksi Redis
- ☐ Queue, worker, dan job processor
- ☐ Graceful shutdown worker
- ☐ Error handling dan logging

Card 25 — JOBS — Booking Expiration

- ☐ Job expiration (delayed 15 menit) dijadwalkan saat booking dibuat
- ☐ Cek status → expire booking unpaid
- ☐ Update status payment bila perlu, lepaskan availability
- ☐ Job idempotent
- ☐ ➕ **Sweeper periodik** (repeatable job) sebagai jaring pengaman jika delayed job hilang

- [ ] ➕ Test race antara expiration job dan webhook
- [ ] Test expiration job

Card 28 — NOTIFICATION — Email Infrastructure

- [ ] Pilih provider (Nodemailer / Resend)
- [ ] Konfigurasi env dan email service
- [ ] Handle kegagalan provider
- [ ] Test pengiriman

Card 29 — NOTIFICATION — Templates

- [ ] Booking dibuat, pembayaran sukses, booking dibatalkan, booking expired, pengingat
- [ ] Template HTML dengan data dinamis
- [ ] Test template

Card 26 + 27 — JOBS — Notifikasi, Reminder & Retry (digabung)

- [ ] Notification queue dan job (booking, payment)
- [ ] Job reminder terjadwal (mis. H-1)
- [ ] Retry attempts dan backoff
- [ ] Track failed job, dead-letter strategy, dan monitoring
- [ ] Log kegagalan
- [ ] Test perilaku retry

F16 — FE — Status Real-time Ringan

- [ ] Refetch on focus / polling status booking
- [ ] Tampilan `EXPIRED` dan slot terbuka kembali
- [ ] (Opsional) Card 30 notifikasi in-app
- [ ] ➕ Cache: refetchOnWindowFocus untuk status booking; polling dihentikan saat status final

Selesai jika: booking yang tidak dibayar kedaluwarsa sendiri dan slotnya terbuka lagi di UI.

Sprint 7 — Dashboard & Review (opsional, batasi waktunya)

Card 33 — DASHBOARD — Owner

- [ ] Revenue metrics, booking metrics
- [ ] Occupancy rate (butuh jam operasional)
- [ ] Resource populer, filter venue
- [ ] Otorisasi dan test

- ☐ ➕ Cek query agregasi dengan `EXPLAIN ANALYZE`

Card 31 — REVIEW — Review & Rating

- ☐ Model Review, rating 1–5
- ☐ `POST /api/bookings/:id/reviews` (hanya booking `COMPLETED` milik user)
- ☐ Cegah review duplikat; update dan delete
- ☐ Moderasi Admin
- ☐ List review, rata-rata rating venue, review per resource
- ☐ Test aturan review

F17 — FE — Dashboard Owner

- ☐ Kartu ringkasan + grafik pendapatan dan okupansi
- ☐ Tabel resource populer
- ☐ ➕ Cache: query dashboard dengan staleTime 1 menit; venueId dan periode ikut menjadi queryKey

F18 — FE — Review

- ☐ Form review dari booking selesai
- ☐ Daftar ulasan dan rating di detail venue
- ☐ ➕ Cache: query review dan rating dengan staleTime 1 menit
- ☐ ➕ Cache: invalidateQueries review dan detail venue (rating rata-rata) setelah kirim/ubah/hapus review

---

FASE C — HARDENING & DELIVERY

Sprint 8 — Hardening

N7 — SECURITY — Rate Limiting

- ☐ Rate limiter berbasis Redis
- ☐ Limit pada login, register, create booking, webhook
- ☐ Response 429 dengan `Retry-After`
- ☐ Test rate limiting

N8 — OPS — Health & Readiness

- ☐ `/health` (liveness) dan `/ready` (cek DB dan Redis)

- [ ] Test endpoint

N9 — OPS — Metrics

- [ ] `prom-client` : latency, request count, error rate
- [ ] Metrik khusus: booking dibuat, konflik 409, job gagal
- [ ] Endpoint `/metrics`
- [ ] ➕ Cache: metrik cache_hits_total dan cache_misses_total (label jenis cache) di prom-client
- [ ] ➕ Cache: metrik cache_errors_total untuk kegagalan Redis
- [ ] ➕ Cache: hit ratio dihitung dari metrik di atas (dipakai di N17)

N10 — SECURITY — Review Keamanan & Performa

- [ ] CORS ketat, batas ukuran body, helmet
- [ ] Audit validasi input dan otorisasi per endpoint
- [ ] Review index database (`EXPLAIN ANALYZE` pada query berat)
- [ ] `npm audit` dan update dependensi

F19 — FE — Polishing

- [ ] Error boundary dan penanganan 429
- [ ] Skeleton dan state kosong konsisten
- [ ] Pass responsif (mobile) dan aksesibilitas dasar

Sprint 9 — Kualitas & Load Test

> **Urutan Sprint 9:** baseline load test (Card 42) → N17 cache → ukur ulang → laporan (Card 44).

Card 39 — TEST — Unit Testing (lengkapi)

- [ ] Price calculation, slot generator, payment verification, state transition
- [ ] Edge cases

Card 40 — TEST — Integration Testing (lengkapi)

- [ ] Auth, Venue, Resource, Availability, Booking, Payment, Review flow

N11 — DATA — Seed Volume Besar

- [ ] Script seed realistis (mis. 100 venue, 500 resource, 10 ribu booking)
- [ ] Gunakan sebagai basis load test

Card 42 — LOAD TEST — Venue & Availability

- [ ] Konfigurasi k6
- [ ] Search venue, filter venue, availability
- [ ] Ukur latency, throughput, error rate
- [ ] ➕ Cache: simpan hasil baseline (p50, p95, p99, throughput) SEBELUM N17 sebagai acuan
- [ ] ➕ Cache: catat skenario k6, data seed, dan spesifikasi mesin agar bisa diulang persis

Card 43 — LOAD TEST — Payment & Webhook

- [ ] Load test create payment dan webhook
- [ ] Concurrent webhook test
- [ ] Ukur latency, throughput, error rate

N17 — CACHE — Redis Cache Layer (Backend)

> Kerjakan **setelah** baseline load test (Card 42) dicatat, supaya ada perbandingan sebelum/sesudah untuk README.

Data	Di-cache?	Catatan
Daftar dan pencarian venue	Ya, TTL ±60 detik	Publik, banyak dibaca, jarang berubah
Detail venue dan resource	Ya, TTL beberapa menit	Hapus saat diubah
Dashboard Owner (agregasi)	Ya, TTL ±60 detik	Query paling berat
Slot template per resource dan tanggal	Boleh	Hanya hasil dari jam operasional
Status booked/diblokir pada availability	Jangan, atau TTL 5-10 detik	Data basi menampilkan slot yang sudah diambil
Membuat booking, status pembayaran	Tidak	Harus selalu dari database
Data per pengguna (Booking saya)	Tidak lewat cache publik	Jangan bocor antar pengguna

> Keamanan data tetap dijamin exclusion constraint database. Cache basi paling buruk membuat pengguna melihat slot yang ternyata sudah diambil (409).

- [ ] Prasyarat: baseline load test (Card 42) sudah dicatat: p50, p95, p99, throughput

- ☐ Helper cacheAside(key, ttl, fn): cek Redis, jika miss ambil dari DB lalu simpan
- ☐ Fallback: Redis error atau timeout tidak boleh menggagalkan request (ambil dari DB dan catat log)
- ☐ Timeout pendek pada operasi Redis agar request tidak menggantung
- ☐ Konvensi key: venues:list:{hash-query}, venue:{id}, resources:venue:{id}, dash:owner:{ownerId}:{venueId}:{periode}
- ☐ TTL per jenis data dengan sedikit jitter (±10%)
- ☐ Invalidasi: venue dibuat/diubah/berubah status, resource diubah, jam operasional atau blokir diubah
- ☐ Pencegahan cache stampede (opsional): satu request mengisi cache, request lain menunggu atau diberi data lama
- ☐ Terapkan pada daftar dan pencarian venue (TTL ±60 detik)
- ☐ Terapkan pada detail venue dan resource (TTL beberapa menit)
- ☐ Terapkan pada dashboard Owner (TTL ±60 detik)
- ☐ Availability: hanya slot template (hasil jam operasional) yang boleh di-cache; status booked/diblokir selalu dari DB atau TTL 5-10 detik
- ☐ Jangan cache: pembuatan booking, status pembayaran, data per pengguna (Booking saya)
- ☐ Pastikan key dan isi cache tidak memuat data per pengguna
- ☐ Metrik cache hit/miss/error (dari N9) dan hit ratio
- ☐ Log hit/miss/error pada level debug
- ☐ Test: hit, miss, invalidasi, TTL kedaluwarsa, dan Redis down (fallback ke DB)
- ☐ Ukur ulang dengan k6 memakai skenario dan data seed yang sama seperti baseline
- ☐ Catat perbandingan p50/p95/p99 dan throughput sebelum vs sesudah
- ☐ Tulis ADR: apa yang di-cache, TTL, strategi invalidasi, dan apa yang sengaja tidak di-cache
- ☐ Masukkan hasil perbandingan ke README (lewat Card 44)

Card 44 — LOAD TEST — Laporan Portofolio

- ☐ Baseline, normal load, stress load
- ☐ p50, p95, p99, throughput, error rate
- ☐ Analisis bottleneck (+ perbaikan dan hasil sebelum/sesudah)
- ☐ Grafik dan kesimpulan
- ☐ ➕ Cache: tambahkan tabel perbandingan p50/p95/p99 dan throughput sebelum vs sesudah N17
- ☐ ➕ Cache: tampilkan cache hit ratio dan jelaskan data apa yang di-cache

F20 — FE — E2E (Playwright)

- ☐ Alur: register → book → bayar
- ☐ Alur: Owner buat venue → Admin approve
- ☐ Alur: konflik slot

- [ ] Jalankan di CI

Sprint 10 — Deployment & Dokumentasi

N12 — DEPLOY — Dockerfile Produksi

- [ ] Dockerfile multi-stage backend dan frontend
- [ ] Compose produksi (API, worker, Postgres, Redis)
- [ ] Env produksi dan manajemen secret

N13 — DEPLOY — Deploy

- [ ] Deploy API + worker, frontend, Postgres, Redis
- [ ] `prisma migrate deploy` saat rilis
- [ ] Domain/URL publik dan HTTPS
- [ ] ➕ Cache: file di folder assets/ (bernama hash dari Vite) memakai Cache-Control: public, max-age=31536000, immutable
- [ ] ➕ Cache: index.html memakai Cache-Control: no-cache
- [ ] ➕ Cache: verifikasi header di DevTools (tab Network) atau dengan curl -I

N14 — CD — Continuous Deployment

- [ ] GitHub Actions deploy otomatis dari `main`
- [ ] Smoke test pasca-deploy (`/ready`)

N15 — DOCS — README & ADR

- [ ] README: deskripsi, fitur, screenshot/GIF, cara menjalankan (`docker compose up`)
- [ ] ERD dan diagram arsitektur
- [ ] ADR: exclusion constraint, idempotency, Redis lock, expiration + sweeper, webhook, kebijakan refund
- [ ] Hasil concurrency test dan load test
- [ ] Bagian "Technical decisions" dan "Known limitations"

Card 45 — DOCS — API Documentation (final)

- [ ] Rapikan OpenAPI semua modul
- [ ] Dokumentasikan error responses dan autentikasi

N16 — CAREER — Finalisasi Portofolio

- [ ] Akun demo per role di README

- [ ] Pin repo di GitHub
- [ ] Tambahkan link live + GitHub ke CV
- [ ] Bullet CV terukur (mis. "100 request konkuren → 1 sukses, p95 = X ms")

BACKLOG (hanya jika masih ada waktu)

Urutan saran: Card 35 → 36 → 37 → 38 (Module 11), lalu Card 30, 32, 34.

Module 11 — Extended Auth

Card 35 — AUTH — Profile & Password

- [ ] `PATCH /api/users/me`
- [ ] Update nama, nomor HP, avatar
- [ ] Ganti password
- [ ] Verifikasi password saat ini
- [ ] Hash password baru
- [ ] Invalidasi session/token yang sesuai
- [ ] Test update profil dan ganti password

Card 36 — AUTH — Account Security

- [ ] Lockout akun sementara
- [ ] Tentukan threshold gagal login dan durasi lockout
- [ ] Lacak percobaan gagal
- [ ] Reset hitungan setelah login berhasil
- [ ] Test lockout
- [ ] ➕ Kombinasikan dengan rate limiter Redis (N7)

Card 37 — AUTH — Email Verification

- [ ] Generate token verifikasi, hash token, simpan expiration
- [ ] Kirim email verifikasi (butuh Card 28–29)
- [ ] Verifikasi token dan tandai email terverifikasi
- [ ] Cegah token dipakai ulang
- [ ] Handle token kedaluwarsa
- [ ] Test alur verifikasi

Card 38 — AUTH — Forgot & Reset Password

- ☐ Endpoint forgot password
- ☐ Generate reset token, hash token, simpan expiration
- ☐ Kirim email reset
- ☐ Endpoint reset password dan hash password baru
- ☐ Invalidasi reset token dan session yang sesuai
- ☐ Handle token kedaluwarsa
- ☐ Test alur reset
- ☐ ➕ Response forgot password sama untuk email terdaftar maupun tidak (cegah user enumeration)

Module 8 — Notification (lanjutan)

Card 30 — NOTIFICATION — In-App

- ☐ Model Notification (Prisma)
- ☐ Notification service
- ☐ Buat notifikasi
- ☐ List notifikasi
- ☐ Tandai dibaca dan tandai semua dibaca
- ☐ Otorisasi (hanya milik sendiri)
- ☐ Test lifecycle notifikasi
- ☐ Frontend: ikon lonceng, daftar notifikasi, badge belum dibaca

Module 10 — Dashboard (lanjutan)

Card 32 — DASHBOARD — Customer

- ☐ Booking mendatang
- ☐ Invoice aktif
- ☐ Riwayat booking
- ☐ Statistik ringkasan
- ☐ Otorisasi dan test dashboard
- ☐ Frontend: halaman dashboard Customer

Card 34 — DASHBOARD — Admin

- ☐ Revenue global
- ☐ Venue menunggu approval
- ☐ User aktif
- ☐ Statistik booking dan platform
- ☐ Otorisasi Admin dan test dashboard

- [ ] Frontend: halaman dashboard Admin

Lain-lain

- [ ] Kebijakan refund otomatis
- [ ] Upload foto venue (object storage)
- [ ] Midtrans/Xendit sandbox sungguhan lewat interface `PaymentProvider`

Ringkasan Dependensi

Sprint	Bergantung pada
1 Venue	0
2 Resource & Availability	1
3 Booking	2
4 Concurrency	3
5 Payment	3, 4
6 Jobs & Notifikasi	5 (Card 28–29 sebelum Card 26)
7 Dashboard & Review	3, 5
8 Hardening	4–6
9 Kualitas & Load Test	5, 8 (N17 setelah baseline Card 42)
10 Deployment & Docs	semua

Jangan dipotong: Sprint 4, 5, 9, dan 10. Empat sprint ini yang membedakan BookSpot dari project tutorial.